RC4 Stream Cipher — Implementation and Cycle Measurements

Cryptography and Security Implementations

1. Implementation (Tasks 1 and 2)

The RC4 Key Scheduling Algorithm (KSA) and Pseudo-Random Generation Algorithm (PRGA) were implemented in C, following Algorithm 1 and Algorithm 2 of the assignment exactly. The complete source is given in Appendix A.

One implementation note: both algorithms require arithmetic modulo 256. Because the state array and the indices *i* and *j* are declared as `uint8_t` (unsigned 8-bit), addition wraps around at 256 automatically. Writing `j = j + S[i]` therefore already computes `(j + S[i]) mod 256`, and no explicit modulo operation is needed.

2. Correctness Check

Before any timing was recorded, the implementation was checked against a published test vector. RFC 6229, *Test Vectors for the Stream Cipher RC4*, lists known keystream bytes for several key lengths. For the 40-bit key 0x0102030405, the first 16 keystream bytes should be:

```
b2 39 63 05 f0 3d c0 27 cc c3 52 4a 0a 11 18 a8
```

The program printed:

```
b2396305f03dc027ccc3524a0a1118a8
```

These match, so the KSA and PRGA are both producing correct RC4 output and the measurements below are measurements of RC4 and not of a broken program.

3. Experimental Measurements (Tasks 3 and 4)

Cycles were counted with the `__rdtsc()` instruction, which reads the processor's timestamp counter directly. The counter is read immediately before and immediately after the code being measured, and the difference is the number of cycles elapsed. Measurements were taken on a Dell Inspiron 3593 running Ubuntu 24.04, compiled with `gcc -O2`.

Quantity	Measured value
KSA, total cycles	12,785 cycles
KSA, cycles per iteration (256 iterations)	49.94 cycles
PRGA, total cycles for 65,536 bytes	799,882 cycles
PRGA, cycles per output byte	12.21 cycles/byte

4. Analysis (Task 5)

4.1 The PRGA figure is the reliable one

Each PRGA iteration performs a small, fixed amount of work: increment *i*, read S[i], update *j*, read S[j], swap the two entries, and read one more entry to produce the output byte. That is roughly eight or nine simple operations, so a cost of about 12 cycles per byte is what one would expect.

The cost per byte does not grow with the amount of output requested. This is because the state is always exactly 256 bytes, no matter how long the message is. The PRGA therefore costs a constant number of cycles per byte, and generating *n* bytes costs time proportional to *n*.

The reason the figure is not smaller is that the operations cannot be overlapped. The value of *j* cannot be computed until S[i] has been read, the swap cannot happen until S[j] has been read, and the output byte cannot be read until the swap is finished. Each step needs the result of the previous step, so the processor must wait for them in order rather than doing several at once. This is an inherent property of RC4, not of this particular program.

4.2 The KSA figure is inflated, and why

Taken at face value the KSA costs 49.94 cycles per iteration. This cannot be correct as a measure of the algorithm itself, and the program's own PRGA measurement is what shows it. A KSA iteration and a PRGA iteration do almost the same work — both update *j* and perform one swap — except that the KSA does *less*, because it does not perform the final lookup to produce an output byte. A KSA iteration must therefore cost slightly *less* than a PRGA byte, not four times more.

The discrepancy comes from how the measurement was taken:

(a) The KSA is timed once, on a single cold run. The 256-byte array S is being written for the very first time, so it is not yet in cache and the processor must fetch it from main memory. The PRGA, measured immediately afterwards, finds S already in cache.

(b) Reading the timestamp counter itself costs some cycles. For the PRGA this fixed cost is divided across 65,536 bytes and becomes invisible; for the KSA it is divided across only 256 iterations and remains visible.

Using the PRGA rate as the guide, the true warm cost of the KSA is roughly 10 cycles per iteration, or about 2,500 to 3,000 cycles in total. The measured 12,785 should be read as an upper bound. A more careful measurement would run the KSA many times and report the median, which removes both effects.

4.3 Setup cost compared with output cost

It is useful to express the key setup cost in units of keystream. At about 2,500 cycles for the KSA and 12.21 cycles per byte, key setup costs roughly the same as generating 200 bytes of keystream.

This has a practical consequence. If a long message is encrypted under one key, the setup cost is negligible and the total time is essentially the PRGA rate. If instead a fresh key is set up for every short message, the setup cost dominates. The second case is also the dangerous one cryptographically: frequent rekeying with related keys is exactly the situation that the known attacks on the RC4 key schedule exploit, and it is why RFC 6229 notes that the initial bytes of the keystream should be discarded.

4.4 Summary

The implementation is correct against the published RFC 6229 test vector. The PRGA costs 12.21 cycles per output byte, constant in the length of the output. The KSA measurement of 12,785 cycles is an upper bound inflated by cold cache and timer overhead; comparison with the PRGA rate indicates a true cost near 2,500 cycles. Key setup is therefore equivalent to a few hundred bytes of keystream.

Appendix A — Source Code (rc4.c)

Compiled and run with:

```
gcc -O2 -Wall -o rc4 rc4.c
./rc4
```

```
/* RC4 stream cipher -- KSA and PRGA, with clock cycle measurements.
 * Algorithms from Boneh & Shoup, "A Graduate Course in Applied Cryptography".
 *
 * Build: gcc -O2 -o rc4 rc4.c
 */

#include <stdio.h>
#include <stdint.h>
#include <x86intrin.h>          /* __rdtsc() */

#define SWAP(a, b) do { uint8_t tmp_ = (a); (a) = (b); (b) = tmp_; } while (0)
static uint8_t S[256];
```

```
static uint8_t i, j;            /* PRGA counters */

/* Algorithm 1: Key Scheduling Algorithm */
void ksa(const uint8_t *key, int s)
{
    int t;
    uint8_t jj = 0;

    for (t = 0; t < 256; t++)
        S[t] = (uint8_t)t;

    for (t = 0; t < 256; t++) {
        uint8_t k = key[t % s];
        jj = jj + S[t] + k;             /* uint8_t wraps == mod 256 */
        SWAP(S[t], S[jj]);
    }
    i = j = 0;
}

/* Algorithm 2: Pseudo-Random Generation Algorithm */
void prga(uint8_t *out, long n)
{
    long t;

    for (t = 0; t < n; t++) {
        i = i + 1;                      /* mod 256 by wraparound */
        j = j + S[i];
        SWAP(S[i], S[j]);
        out[t] = S[(uint8_t)(S[i] + S[j])];
    }
}

int main(void)
{
    static uint8_t out[65536];
    static const uint8_t key[5] = { 0x01, 0x02, 0x03, 0x04, 0x05 };
    uint64_t start, ksa_cycles, prga_cycles;
    long n = 65536;
    int t;

    /* (3) total clock cycles for the KSA */
    start = __rdtsc();
    ksa(key, sizeof key);
    ksa_cycles = __rdtsc() - start;

    /* (4) clock cycles per output byte for the PRGA */
    start = __rdtsc();
    prga(out, n);
    prga_cycles = __rdtsc() - start;

    printf("KSA  : %llu cycles total (%.2f per iteration)\n",
           (unsigned long long)ksa_cycles, ksa_cycles / 256.0);
    printf("PRGA : %llu cycles for %ld bytes (%.2f cycles/byte)\n",
           (unsigned long long)prga_cycles, n, (double)prga_cycles / n);

    printf("\nFirst 16 keystream bytes: ");
    for (t = 0; t < 16; t++)
        printf("%02x", out[t]);
    printf("\n");

    return 0;
}
```